TreeSync Manual

TreeSync syncs two directory trees and checks every copy by SHA-256. It has a GUI (`treesync`) and a command-line program (`treesync-cli`) that does the actual work; the GUI starts `treesync-cli` and shows what it does.

This manual takes you from a fresh Linux machine to a running sync:

Part	What it covers
1. Bootstrap	Get the code, install the compilers and libraries, get U++, first build
2. Build	`build.sh`, the Build page, debug and release, tests, `verify.sh` and the pre-push hook
3. Run	The GUI step by step, the command line, remote trees, root, checking and attributes
4. Files and troubleshooting	Where everything is kept, and what to do when something goes wrong

> **Warning.** TreeSync is experimental testing software. It copies, overwrites and deletes files. Try it on copies first, keep backups, and use **Dry run** before a real run. It is provided "AS IS" under the Apache License 2.0 (sections 7 and 8).

In TreeSync, the **Help** menu opens these chapters as PDFs, and **F1** opens the one for the window you are in (main window and Setup: Run; Build page: Build).

Rebuilding the PDFs. The `.md` files here are the sources. After editing them run `./build-pdf.sh` in this folder: it writes `pdf/`, one PDF per chapter and `TreeSync-Manual.pdf` with all of them (it needs python3 with its markdown module, Chrome or Chromium, and pdfunite).

Quick start

For someone who has done this before. Each step is explained in the parts above.

```
# 1. Get the code (the branch in use is dev-work)
git clone ssh://git@192.168.1.26:2222/jay/jac317.git
cd jac317/v001

# 2. Check the machine, install what is missing (it asks first), build and test
./bootstrap.sh

# 3. Build the release programs the GUI runs (with all tests, both compilers)
./verify.sh

# 4. Start the GUI
out/gcc/treesync
```

In the GUI: pick a **source** and a **target** folder, check what to sync in the source tree, then **Sync > Dry run** (Shift+F9) to see what would happen and **Sync > Run** (F9) to do it.

Words used in this manual

Word	Meaning
Source	The tree copied from
Target	The tree copied to
Copy / Mirror / Merge	The three modes: add and update only; make the target match the source (removing extras); copy changes both ways
Dry run	A run that only reports what it would do and changes nothing
Safe delete	Instead of deleting or overwriting, TreeSync moves the old file into `<target>.old` (on by default)
Run	One sync, with a number; it is recorded in the database and can be resumed if stopped
Scan only	Reads and hashes both trees and compares them; changes nothing
`v001/`	The folder that holds the code, the scripts and (after a build) `out/` with the programs

1. Bootstrap: from a fresh machine to a first build

`v001/bootstrap.sh` takes a Linux machine from nothing to a built and tested TreeSync. It is safe to run again at any time: whatever is already there is left alone.

1.1 What you need

- **Linux**: RHEL 9 and its rebuilds (Rocky, Alma), Fedora, or Linux Mint / Ubuntu / Debian. TreeSync is Linux only for now.
- **A user that can use `sudo`**, for installing packages (bootstrap shows every command and asks before it runs any).
- **Network access**, to install packages and, if U++ is not on the machine yet, to download it.
- About **2 GB of disk** for U++ and the build output.

1.2 Get the code

```
git clone ssh://git@192.168.1.26:2222/jay/jac317.git
cd jac317
git checkout dev-work      # the branch in use
cd v001
```

Everything below runs from `jac317/v001`.

1.3 See what bootstrap would do (optional)

```
./bootstrap.sh --dry-run
```

This checks the machine and prints the plan: which requirements are present, the install command for each missing one, and whether U++ would be fetched. Nothing is installed or built.

1.4 Run bootstrap

```
./bootstrap.sh
```

It works in five steps.

Step 1: the Linux family

It reads `/etc/os-release` and picks the install recipes for `rhel`, `fedora` or `linuxmint` (Ubuntu and Debian count as `linuxmint`).

Step 2: the requirements

Each requirement in `setup/recipes.conf` is checked and listed as `ok` or `MISSING`:

Name	What it is	Needed for
`gcc`	GCC 15 or newer (RHEL: `gcc-toolset-15`)	building (required)
`clang`	Clang 21 or newer	building (required)
`gui`	GTK 3 and the libraries U++ GUIs build with, OpenSSL for SSH	building (required)
`tools`	make, git, curl, tar, xz	fetching and building U++ (required)
`uppfix`	A one-line fix to U++'s heap (gcc release builds crash without it)	building (required)
`testing`	bc and OpenSSH client and server	the compiler comparison and the SFTP tests (optional)
`checks`	AddressSanitizer + UBSan runtimes and valgrind	the checked test runs (optional)

For the missing ones it shows the install commands, for example on RHEL:

```
To install (these change the system; most need sudo):
  gcc: sudo dnf install -y gcc-toolset-15
  gui: sudo dnf install -y gtk3-devel libnotify-devel ...

Run these now? [y/N]
```

Answer `y` to run them (you will be asked for your `sudo` password), or `n` to stop and run them by hand. If a required one is still missing afterwards, bootstrap stops here.

Step 3: U++

TreeSync is built with U++ (its build tool `umk` and its sources `uppsrc`). Bootstrap looks for one in this order:

1. `$UPP_DIR` (if you set it)
2. The shared copy on the GFS disk: `/run/media/jay/GFS/upp` (or `/media/jay/GFS`, `/mnt/GFS`) with `upp-bin/rhel/umk` or `upp-bin/mint/umk`
3. `~/u++`

If none is found it offers to download the latest U++ release into `~/u++` and build its `umk` there. This needs no `sudo`. To put it somewhere else:

```
./bootstrap.sh --upp-dir /opt/u++
export UPP_DIR=/opt/u++      # then build.sh finds it too (put this in ~/.bashrc)
```

Step 4: build and test

For each compiler it runs `./build.sh --compiler <gcc|clang> --test`: a **debug** build of the three programs and the core tests. The output of each goes to `out/bootstrap-gcc.txt` and `out/bootstrap-clang.txt`.

It also points git at the project's pre-push hook (`git config core.hooksPath v001/setup/hooks`), so every `git push` first runs the full check (see 2.5).

Step 5: summary

It ends with one line per step in the start : stop : duration layout:

```
 Start    : Stop     : Duration         : Command / Comments
 14:02:58 : 14:03:01 : 000:000:00:00:03 : Check requirements........ : Result = ....0 : Good
 14:03:01 : 14:05:40 : 000:000:00:02:39 : Build and test with gcc... : Result = ....0 : Good
: passed: 290/290 checks
```

The same lines are added to `Bootstrap-Results.txt` in TreeSync's logs folder.

1.5 Bootstrap options

Option	What it does
`--dry-run`	Only say what it would do
`--yes`	Do not ask before installing (for scripts)
`--compiler gcc` / `clang` / `both`	Build with one compiler (default both)
`--no-build`	Stop after the checks and installs
`--upp-dir DIR`	Where U++ goes if it has to be fetched (default `~/u++`)
`--check`	Only check, one `@req` line per requirement (the Build page uses this); exit 1 if a required one is missing
`-h`, `--help`	Show the help

1.6 After bootstrap

Bootstrap leaves **debug** builds in `out/gcc/` and `out/clang/`. For everyday use build the faster **release** programs, which is what the GUI should run:

```
./verify.sh          # debug and release, all tests, both compilers; ends with release builds
```

Then go on to 3. Run.

2. Build

All building goes through `v001/build.sh`; the GUI's Build page, `verify.sh` and `bootstrap.sh` all call it. Run everything from `jac317/v001`.

2.1 What gets built

Program	What it is
`treesync`	The GUI
`treesync-cli`	The command-line program; does every sync (the GUI starts it)
`treesync-test`	The core tests (built with `--test`)

Each compiler has its own folder: `out/gcc/` and `out/clang/`. The two builds are the same program; TreeSync is also a proving ground for GCC 15 and Clang 21 with C++26.

2.2 build.sh

```
./build.sh --compiler gcc                # debug build with gcc 15
./build.sh --compiler clang              # debug build with clang 21 (the default compiler)
./build.sh --compiler gcc --release      # optimized build: use this for real syncs
./build.sh --compiler gcc --test         # build, then run the core tests
./build.sh --compiler gcc --release --test
./build.sh --compiler gcc --clean        # compile everything, not only what changed
```

Option	What it does
`--compiler gcc` / `clang`	Which compiler (default clang)
`--release`	Optimized build (default: debug)
`--test`	Also build `treesync-test` and run it; the exit code is non-zero if a test fails
`--clean`	Rebuild everything (fair build times when comparing compilers)
`--tsan`	Instead of the build: the tests under ThreadSanitizer (clang), into `out/clang-tsan/`
`--asan`	The tests under AddressSanitizer + UBSan, then `treesync-cli --bench` under them
`--valgrind`	The tests under valgrind memcheck (slow)
`--helgrind`	The tests under valgrind helgrind (very slow)

Debug or release? Debug builds check more and are easier to debug, but are much slower: on many small files about 4 times the CPU. Always run real syncs with a **release** build.

`build.sh` remembers which type is in each `out/` folder and links the programs again when you switch, so a debug build never leaves a release program behind or the other way round.

A test run ends with a line such as `passed: 290/290 checks`. A debug build skips one test (a 4.5 GiB sparse copy, too slow without optimization) and says so; a release test run has all of them (`passed: 303/303 checks`).

2.3 The Build page in the GUI

File > Build... opens it, and it is also the first screen when TreeSync starts (untick **Show this page when TreeSync starts** to go straight to the main window).

- **Compiler**: gcc, clang, or both one after the other.
- **Type**: Debug or Release.

- **Run the tests (treesync-test) after building**: the one switch for testing. Unticked, **Build** only builds. Ticked, the tests run, plus the checked runs below it:

    - Also under ThreadSanitizer (clang; data races)
    - Also under AddressSanitizer + UBSan (memory errors, undefined behaviour)
    - Also under valgrind memcheck (slow)
    - Also under valgrind helgrind (very slow)

  These keep their ticks while "Run the tests" is off; they are greyed out and skipped. - **Clean build**: compile everything. - **Compare the compilers**: versions, C++26 features, `import std`, speed and size, side by side. - **Display**: Wayland (recommended) or X11. - **This machine**: whether the requirements are present (**Check again**, **Open terminal to fix...**).

The command lines it will run are shown at the bottom. **Build** runs them one after another and shows each one's output; the results screen sums them up. **Continue to TreeSync using GCC exe / Clang exe** starts the main window with that build.

Your choices are saved in `~/.config/treesync/build`.

2.4 The sanitizer and valgrind runs

These find bugs the normal tests cannot see: data races between the copy threads, memory errors, undefined behaviour. They take much longer than the normal tests, so run them when threading or memory handling changed:

```
./build.sh --compiler clang --tsan
./build.sh --compiler gcc --asan
./build.sh --compiler gcc --valgrind
./build.sh --compiler gcc --helgrind
```

They need the optional `checks` requirement (see 1.4). Each builds into its own folder (`out/gcc-asan/` and so on) and never touches the programs in `out/gcc/`.

2.5 verify.sh: the full check, and the pre-push hook

```
./verify.sh                 # gcc and clang
./verify.sh --compiler gcc  # one compiler
```

For each compiler it does a **debug build with every test**, then a **release build with every test**. It stops at the first failure and ends with **release builds in `out/`**, the programs the GUI runs. Each step's full output is in `out/verify/<compiler>-<type>.log`. It also checks that each step really tested the type it claims. A run takes well under a minute when little has changed:

```
verify: debug, then release; every test; compilers: gcc clang
 gcc    debug    building and testing ... ok    10 s  passed: 296/296 checks
 clang  debug    building and testing ... ok     7 s  passed: 296/296 checks
 gcc    release  building and testing ... ok    13 s  passed: 303/303 checks
 clang  release  building and testing ... ok    13 s  passed: 303/303 checks
 took 43 s
 verify: all good; out/ holds release builds
```

The pre-push hook. `v001/setup/hooks/pre-push` runs `verify.sh` before every `git push`; a failure stops the push. `bootstrap.sh` sets it up; by hand:

```
git config core.hooksPath v001/setup/hooks
```

To push once without the check: `git push --no-verify`.

2.6 After changing code

1. Build and test while working: `./build.sh --compiler gcc --test`.
2. Before pushing (or let the hook do it): `./verify.sh`.
3. When threads, locks or memory handling changed, also the sanitizer runs (2.4).

`verify.sh` leaves release builds in `out/`, so the GUI is fast again afterwards.

3. Run

3.1 Start the GUI

```
cd jac317/v001
out/gcc/treesync          # or out/clang/treesync
```

Run the **release** build (see 2. Build); `./verify.sh` leaves one in `out/`.

If **Show this page when TreeSync starts** is ticked, the Build page comes first: click **Continue to TreeSync using GCC exe** (or Clang) to go on, or **Build** first.

Options on the command line set up the window and become the saved settings:

```
out/gcc/treesync --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror
```

Without options, the settings saved last time are used (`~/.config/treesync/settings`, saved on every change).

Run as yourself, not with sudo. The GUI is an ordinary desktop program. When a sync needs root, use **Sync > Run as root** (3.9); only the sync itself runs as root.

3.2 The main window

```
+-------------------------------------------------------------------------------+
| File  View  Sync                    Elapsed 0:12  Total ~0:16  Done 03:34     |
+-----------------------------------+-----------------------------------+
| Disk  [ nvme0n1 ...          v]   | Disk  [ sdb ...                 v]   |
| Mount [ /home/jay            v]   | Mount [ /run/media/jay/Backup   v]   |
| Path  [ /home/jay/Photos ] [...]  | Path  [ .../Backup/Photos ] [...]    |
| [-] [x] Source: /home/jay/Photos  | [-] Target: .../Backup/Photos        |
|     [+] [x] 2024                  |     [+] 2024                         |
|     [+] [-] 2025                  |                                      |
|         [ ] notes.txt             |                                      |
+-----------------------------------+-----------------------------------+
| Pre-scan: source        | Copy                  | Post-scan: target       |
| [==========      ]      | [=====          ]     | [===             ]      |
+-------------------------------------------------------------------------------+
| Copying                              Rate       Size     Time              |
| [ 12/140] 2024/IMG_0001.jpg  [====   ]   85 MiB/s   130 MiB  0:01            |
+-------------------------------------------------------------------------------+
| log: one line per copied file, warnings, the summary                          |
+-------------------------------------------------------------------------------+
| status bar: the command line the current settings stand for                   |
+-------------------------------------------------------------------------------+
```

- **Left: the source**, **right: the target**. Each side has a **Disk** and **Mount** list to jump to a drive, a **Path** box (type a path and press Enter, or **...** to browse), and the tree below it.
- **Trees**: click [+] to open a folder, or use the + and - keys. **View > Reload trees** reads them again, **View > Collapse all** closes them.

- **The progress boxes** and the list of files in progress appear while a run is going. The upper right of the menu bar shows how long it has run, how long the whole run will probably take, and when it should be done.
- **The log** shows what happened; **View > Show log file** (Ctrl+L) opens the full log of the newest run.

Without root, local paths are limited to your home folder; the status bar says so.

Help opens this manual: the whole of it, or one chapter, as a PDF. **F1** opens the chapter for the window you are in.

3.3 Choose what to sync: the checkboxes

Every folder and file in the **source** tree has a checkbox.

- **Check a folder**: everything below it is synced, including files added there later.
- **Uncheck a folder**: nothing below it is synced.
- **Uncheck one file** inside a checked folder: the rest of that folder stays checked.
- A folder with only some of its contents checked shows a **dash** (partly checked).
- The **top box** checks or unchecks the whole tree.

What is checked is saved as `--include-path` settings. Folders above a checked item are created on the target only where something checked goes, and **Mirror never deletes anything outside the checked paths**. Changing the source folder starts again with everything checked. If nothing is checked, Run refuses to start.

3.4 Setup: mode and options

File > Setup... has three tabs.

Source tree / Target tree: the path, and for a remote machine the **Remote** box with **Host**, **Port**, **User** and **Key file** (see 3.10).

Flags:

- **Mode**:
  - **Copy**: new and changed files go from source to target; nothing on the target is removed. Turns on *Skip older*: a newer file on the target is not overwritten.
  - **Mirror**: the target is made to match the source; files only the target has are removed (with safe delete: moved to `<target>.old`).
  - **Merge**: changes go both ways; a file changed on both sides since the last sync is a conflict and is left alone. Nothing is removed.
- **Reset to the defaults** of Copy, Mirror or Merge.
- **Owners**: don't copy / by name (normal) / by number. Setting an owner needs root.
- **Switches**: click any option to see what it does in the help box. The important ones: *Safe delete* (on; keep it on), *Hard links*, *Follow links*, *Skip older / newer / changed*, *Keep extra*, *Wipe target first*, *Low priority*.
- **Numbers**: threads (default 4), retries, chunk size, bandwidth limit, how long to wait for a lost network, time tolerance.

- **Include / exclude**: file and folder names with `*` and `?`, separated by `;`. A blank box filters nothing; exclude wins over include.

3.5 Dry run, then run

1. **Sync > Dry run** (Shift+F9): lists what a run would do (`copy`, `link`, `delete`, `attrs`, `mkdir`) and changes nothing. Read it.
2. **Sync > Run** (F9): asks for confirmation, saying what the mode will do and where removed and replaced files go, then runs.

While it runs:

- **Pre-scan: source** walks the source. **Copy** shows the bytes copied of the total. **Post-scan: target** checks each target folder for files to remove (Mirror).
- Each file is written to `NAME.treesync-part`, read back, checked against the source's SHA-256, given the source's permissions, times, owner and extended attributes, and only then renamed into place.
- A file whose content is the same but whose permissions or time changed gets only its attributes set (an `attrs` step), without copying.

At the end a popup says how it went and how long it took. **Sync > Stop** (Esc) stops after the current file or chunk.

3.6 Stop and resume

A stopped run (Stop, Ctrl+C in a terminal, a crash, a reboot, an unplugged disk) is **resumed by running it again with the same settings**. Every step was recorded in the database, so it continues where it was. A big file continues at its last good chunk: on resume, every chunk already copied is read back and checked first, and from the first damaged one the file is copied again.

To start over instead, use `--fresh` on the command line.

3.7 Checking and attributes

- **Sync > Scan only (SHA-256 check)** (F7): changes nothing. Lists both trees, reads and hashes **every file on both sides** afresh, and compares permissions, times, owners and extended attributes. Each file's log line ends in `same`, `DIFFERS`, `missing` (not on the target) or `failed`. Use it after a big copy to prove the target is right.
- **Sync > Sync attributes** (F8): sets only the attributes that differ (mode, time, owner, extended attributes) on the target, for files of the same size and for folders. No content is copied.
- **Sync > Clear .old folders...**: deletes the safe-delete folders (`<target>.old`) for good, after a confirmation that says how many files and bytes. Folders made by a root run need **Run as root**.

3.8 History, logs and AI export

- **View > History...** (Ctrl+H): every run, newest first, with its counts (folders, copied, deleted, attributes set, failed, conflicts, bytes). Failed runs are red. Below:
  - **Actions**: what the selected run did, failures first.
  - **SHA-256 checks**: every checked file under a line naming its folder, with the source and target SHA-256 side by side, whether they match, and how the attributes compare. Mismatches first, in red; attribute differences in orange.
  - **Show log**, **Export logs to AI.md**, **Export SQL to AI.md**.
- **Logs**: each run writes `logs/YYYY-MM-DD_HHMMSS.log` next to the database. It starts with a header; every line has its start, stop and duration to the microsecond.
- **View > Export newest run for AI**: writes the run as Markdown files into `logs/ai/` (a guide, problems, slowest copies, throughput, the logs in full; or the database contents), ready to give to an AI to analyse.

3.9 Running as root

A sync often meets files your user cannot read or replace (other users' files, system files). Without root, local trees must be inside your home folder.

- **In the GUI**: tick **Sync > Run as root (pkexec)**. Each run then starts `treesync-cli` through `pkexec`, which asks for your password; the GUI itself stays an ordinary program.
- **In a terminal**: `sudo out/gcc/treesync-cli ...`.

A root run keeps its own database and logs under `/root/.local/share/treesync/`, separate from yours. To resume a root run, run it as root again.

3.10 Remote trees (SFTP)

Either side can be on another machine over SSH. One-time setup:

```
ssh-keygen -t ed25519            # if you have no key yet
ssh-copy-id jay@server           # passwordless login
ssh jay@server                   # once, to check and accept the host key
```

TreeSync logs in with keys only, never passwords. In the GUI: **File > Setup > Target tree**, tick **Remote**, and fill in Host, Port, User (and Key file if not the usual one). On the command line: `--target-host=server --target-user=jay`.

If the network drops, the run waits (default up to 30 minutes, *Wait for the network*), shows **NETWORK UNAVAILABLE**, and carries on when the machine is back.

3.11 The command line

`treesync-cli` takes the same options the GUI saves. **It does a dry run unless you give `--no-dry-run`.**

```
# What would a mirror do?
out/gcc/treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror

# Do it
out/gcc/treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror
--no-dry-run

# Copy only new and changed files, 8 threads, at low priority
out/gcc/treesync-cli --source=/data --target=/backup/data --copy --threads=8 --low-priority -
-no-dry-run

# Only one folder and one file of the source
out/gcc/treesync-cli --source=/data --target=/backup/data --include-path=projects/2026 --
include-path=notes.txt --no-dry-run

# To a remote machine
out/gcc/treesync-cli --source=/data --target-host=server --target-user=jay --
target=/backup/data --no-dry-run

# Prove a copy: hash both sides, change nothing
out/gcc/treesync-cli --source=/data --target=/backup/data --scan-only

# Set only differing permissions and times
out/gcc/treesync-cli --source=/data --target=/backup/data --sync-attrs --no-dry-run

# Delete <target>.old for good (without --no-dry-run it only counts)
out/gcc/treesync-cli --source=/data --target=/backup/data --clear-old --no-dry-run
```

`treesync-cli --help` lists every option. In a terminal it shows one line per copied file, `[ 6/90] name 100% | 85.3 MiB/s | 130.9 MiB | 00m01s`, and Ctrl+C stops it (press twice to quit at once).

Exit codes: 0 in sync (or, for Scan only, the trees are the same); 1 failures, conflicts or differences; 2 could not run (bad options, a path refused); 3 stopped.

Database views for looking at results directly:

```
sqlite3 ~/.local/share/treesync/treesync.db
```

```
SELECT run, started, state, copied, failed FROM run_summary ORDER BY run DESC LIMIT 10;
SELECT * FROM failures WHERE run = 42;
SELECT path, file_name, problem FROM hash_mismatches WHERE run = 42;
SELECT path, file_name, attr_diff, result FROM attr_mismatches WHERE run = 42;
```

4. Files and troubleshooting

4.1 Where everything is kept

What	Where
The programs	`v001/out/gcc/` and `v001/out/clang/`: `treesync`, `treesync-cli`, `treesync-test`
Build and test output	`v001/out/` (`verify/` logs, `bootstrap-*.txt`); ignored by git
GUI settings	`~/.config/treesync/settings`: one option per line, as on the command line
Build page choices	`~/.config/treesync/build`
Database	`~/.local/share/treesync/treesync.db` (SQLite): runs, actions, hash cache, checks
Run logs	`~/.local/share/treesync/logs/YYYY-MM-DD_HHMMSS.log`
Results of every run	`~/.local/share/treesync/logs/Results.txt`, one line per run
AI exports	`~/.local/share/treesync/logs/ai/run-N-logs.AI.md`, `run-N-sql.AI.md`
Root runs	the same under `/root/.local/share/treesync/`
Safe-delete folder	`<target>.old` next to the target (or `--safe-delete-dir`)
Unfinished copies	`NAME.treesync-part` next to where the file goes; removed when done
This manual	`Manual/*.md` (the sources) and `Manual/pdf/` (built by `Manual/build-pdf.sh`)

`--db=PATH` puts the database (and its `logs/` folder) somewhere else.

4.2 Troubleshooting

"TreeSync is not running as root ... only inside your home directory"
: Local trees outside your home need root: tick **Sync > Run as root**, or run `sudo treesync-cli ...`. Inside your home nothing is needed.

A path is refused
: Without root, a path that leaves your home (also through `..` or a symlink) is refused. Use root, or a path inside your home.

bootstrap stops: "Still missing: ..."
: A required package did not install. Run the install command it printed by hand and read its error (often a repository that is not enabled, for example CodeReady Builder on RHEL for `gtk3-devel`), then run `./bootstrap.sh` again.

build.sh: "U++ (umk + uppsrc) not found"

Run `./bootstrap.sh`, or point to an existing U++ with `export UPP_DIR=/path/to/u++`.

A gcc release build crashes at start

The U++ heap fix is missing (`uppfix`); `./bootstrap.sh` applies it.

A test fails

`./verify.sh` stops and prints the failing checks; the full output is in `v001/out/verify/<compiler>-<type>.log`. `git push --no-verify` pushes anyway, but fix it first if you can.

A run is slow

Make sure the GUI runs a **release** build (`./verify.sh` leaves them in `out/`). Many small files: more threads (`--threads=8`) can help. Over SFTP the network is usually the limit; `--compress` helps text, not media.

"no Wayland session found" on the Build page

The GUI was started without `WAYLAND_DISPLAY`, usually through `sudo`. Start it as yourself and use **Sync > Run as root** for root runs.

The host key is refused (remote trees)

Connect once with `ssh user@host` to check and accept the key; TreeSync never accepts an unknown or changed key by itself.

A run was stopped and the target has `.treesync-part` files

Run the same sync again: it resumes, re-checks what was already copied, and removes the part files when done. `--fresh` starts over instead.

The `.old` folder keeps growing

Safe delete never deletes. **Sync > Clear .old folders...** (or `treesync-cli --clear-old --no-dry-run`) empties it when you are sure.

Something else

View > Export newest run for AI and give the files in `logs/ai/` to an AI, or read them yourself: they list every problem of the run first.